

Kubernetes Architecture – Detailed Flow Diagram

Below is a **clear flow diagram + step-by-step explanation** of how Kubernetes works internally, from `kubectl` command to Pod running.

1 High-Level Flow Diagram

```
flowchart LR
    User[Admin / Developer  
kubectl] -->|REST API| APIServer[API Server]

    APIServer -->|Read/Write| ETCD[(etcd  
Key-Value Store)]

    APIServer --> Scheduler[Kube Scheduler]
    Scheduler -->|Pod assignment| APIServer

    APIServer --> Controller[Controller Manager]
    Controller -->|Desired = Actual| APIServer

    APIServer --> Kubelet1[Kubelet  
(Node 1)]
    APIServer --> Kubelet2[Kubelet  
(Node 2)]

    Kubelet1 --> Runtime1[Container Runtime  
(Docker / CRI-O)]
    Kubelet2 --> Runtime2[Container Runtime  
(Docker / CRI-O)]

    Runtime1 --> PodA[Pod A]
    Runtime1 --> PodB[Pod B]

    Runtime2 --> PodC[Pod C]
    Runtime2 --> PodD[Pod D]

    KubeProxy1[Kube-Proxy] --> Service1[Service / Networking]
    KubeProxy2[Kube-Proxy] --> Service1
```

2 Control Plane Components (Master Node)

♦ API Server (Brain of Kubernetes)

- Entry point of the cluster
- All components talk **only via API Server**

- Validates requests (auth, admission controller)

◆ etcd

- Distributed key-value database
- Stores:
 - Pod definitions
 - Node info
 - ConfigMaps, Secrets
 - **Single source of truth**

◆ Kube Scheduler

- Decides **which Pod runs on which Node**
- Uses factors:
 - CPU & Memory
 - Taints / Tolerations
 - Node affinity

◆ Controller Manager

- Ensures **desired state = actual state**
 - Controllers:
 - ReplicaSet Controller
 - Node Controller
 - Job Controller
-

3 Worker Node Components

◆ Kubelet

- Node agent
- Responsibilities:
 - Talks to API Server
 - Creates Pods
 - Monitors container health

◆ Container Runtime

- Runs containers
- Examples:
 - Docker
 - containerd
 - CRI-O

◆ Kube-Proxy

- Networking component
- Handles:
 - Service IP
 - Load balancing

- iptables / IPVS rules
-

4 End-to-End Pod Creation Flow

Step-by-Step Execution

1 Developer runs:

```
kubectl apply -f pod.yaml
```

2 Request goes to **API Server**

3 API Server stores config in **etcd**

4 Scheduler finds best Node

5 API Server informs **Kubelet**

6 Kubelet pulls image & creates container

7 Container runtime starts Pod

8 Kube-Proxy enables networking

✓ Pod is now RUNNING

5 Simple Interview Summary (Must Remember)

- API Server = Brain
 - etcd = Database
 - Scheduler = Decision maker
 - Controller = Auto-healing
 - Kubelet = Node manager
 - Kube-proxy = Networking
-

If you want: - **AWS EKS mapping** - **Real-time production example** - **Interview questions with answers**

Tell me ♀